Solution summary

This notebook uses weather data from the Norwegian Meteorological Institute for station SN17850 in Ås. It downloads daily temperature and precipitation for 2025, compares the year with a 1925 record, detects heatwaves using the 5-day threshold, and calculates a 7-day moving average. The results are presented as plots and summary statistics to match the deliverable requirements.
GitHub Repo - INF201_deliverable_1

Imports

python

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import requests
import statistics as st
import os
```

Task 1

Task 1: This section requests daily mean air temperature for all of 2025, plots the annual seasonal pattern, and prints the year's summary statistics. The result shows the expected temperature cycle across the year, with the minimum, maximum, median, and mean values giving a compact view of the climate in Ås for 2025.

python

```
client_id = os.environ.get('client_id')

endpoint = 'https://frost.met.no/observations/v0.jsonld'
parameters = {
'sources': 'SN17850',
'elements': 'mean(air_temperature P1D)',
'referencetime': '2025-01-01/2025-12-31',
}

frost_response = requests.get(endpoint, parameters, auth=(client_id, "
assert frost_response.status_code == 200

frost_data_task1 = frost_response.json()
```

python

```
temp_task1 = []
time_task1 = []

for entry in frost_data_task1['data']:

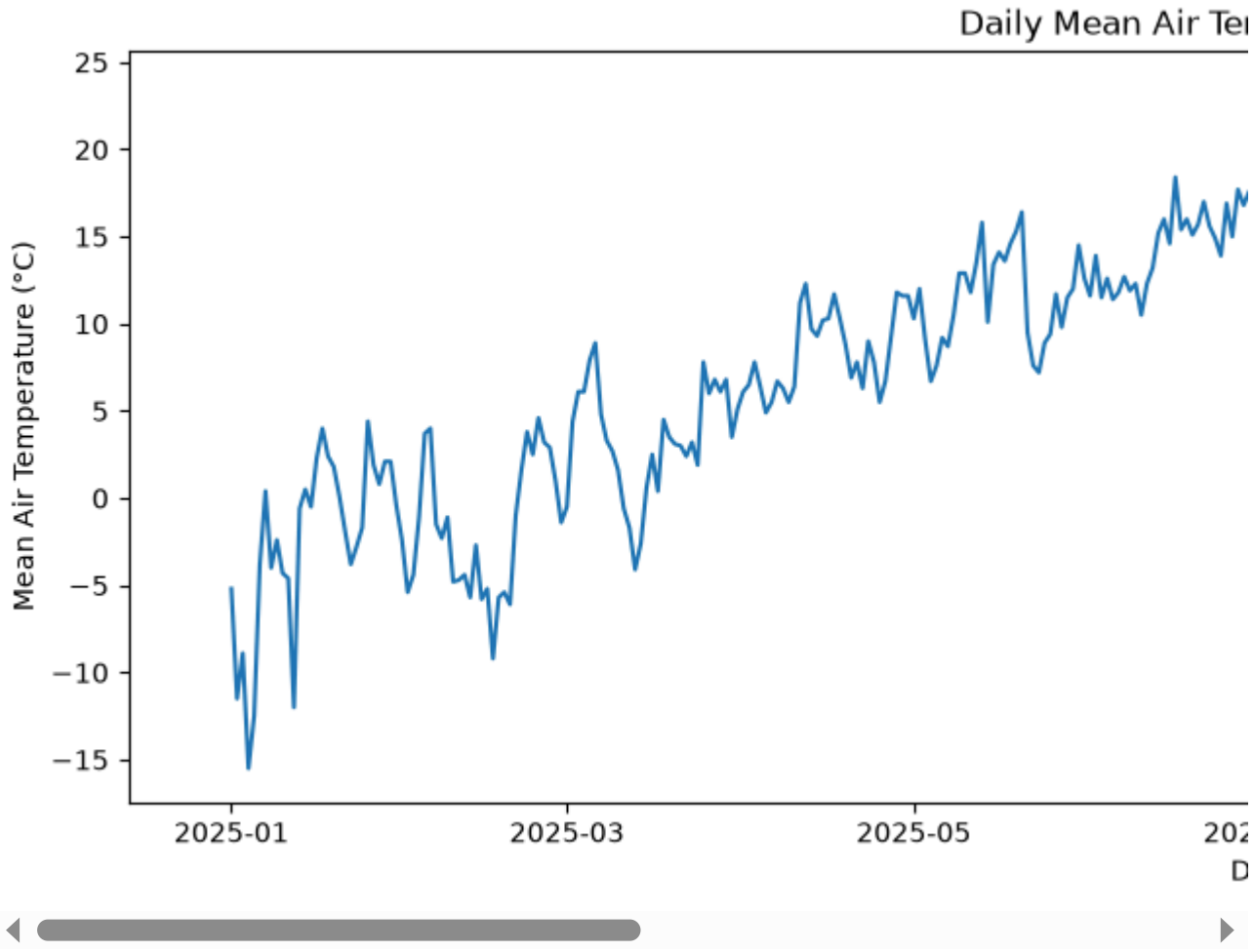
    temperature = entry['observations'][0]['value']
    date = pd.to_datetime(entry['referenceTime'])

    temp_task1.append(temperature)
    time_task1.append(date)
```

Temp. Plot

python

```
plt.figure(figsize=(15, 5))
plt.title('Daily Mean Air Temperature for 2025')
plt.xlabel('Date')
plt.ylabel('Mean Air Temperature (°C)')
plt.plot(time_task1, temp_task1)
plt.show()
```



Summary Table

python

```python
mean_temp_task1 = sum(temp_task1) / len(temp_task1)
max_temp_task1  = max(temp_task1)
min_temp_task1  = min(temp_task1)
median_temp_task1 = st.median(temp_task1)

print(f"Summary Statistics for Daily Mean Air Temperature in 2025:")
print(f"Mean: {mean_temp_task1:8.2f} °C")
print(f"Max: {max_temp_task1:8.2f} °C")
print(f"Min: {min_temp_task1:8.2f} °C")
```

```
Summary Statistics for Daily Mean Air Temperature in 2025:
Mean:     7.90 °C
Max:    23.70 °C
Min:   -15.50 °C
```

Task 2

Task 2: This section downloads daily precipitation for the same period and combines it with the temperature series in a single dual-axis plot. The result makes it easier to see how precipitation events overlap with warm and cold periods, highlighting that temperature and rainfall do not necessarily follow the same seasonal pattern.

python

```python
endpoint = 'https://frost.met.no/observations/v0.jsonld'
parameters = {
'sources': 'SN17850',
'elements': 'sum(precipitation_amount P1D)',
'referencetime': '2025-01-01/2025-12-31',
}

frost_response = requests.get(endpoint, parameters, auth=(client_id, "
assert frost_response.status_code == 200

frost_data_task2 = frost_response.json()
```

python

```python
time_task2 = []
precipitation_task2 = []
for entry in frost_data_task2['data']:

    precip = entry['observations'][1]['value']
    date = pd.to_datetime(entry['referenceTime'])

    precipitation_task2.append(precip)
    time_task2.append(date)
```

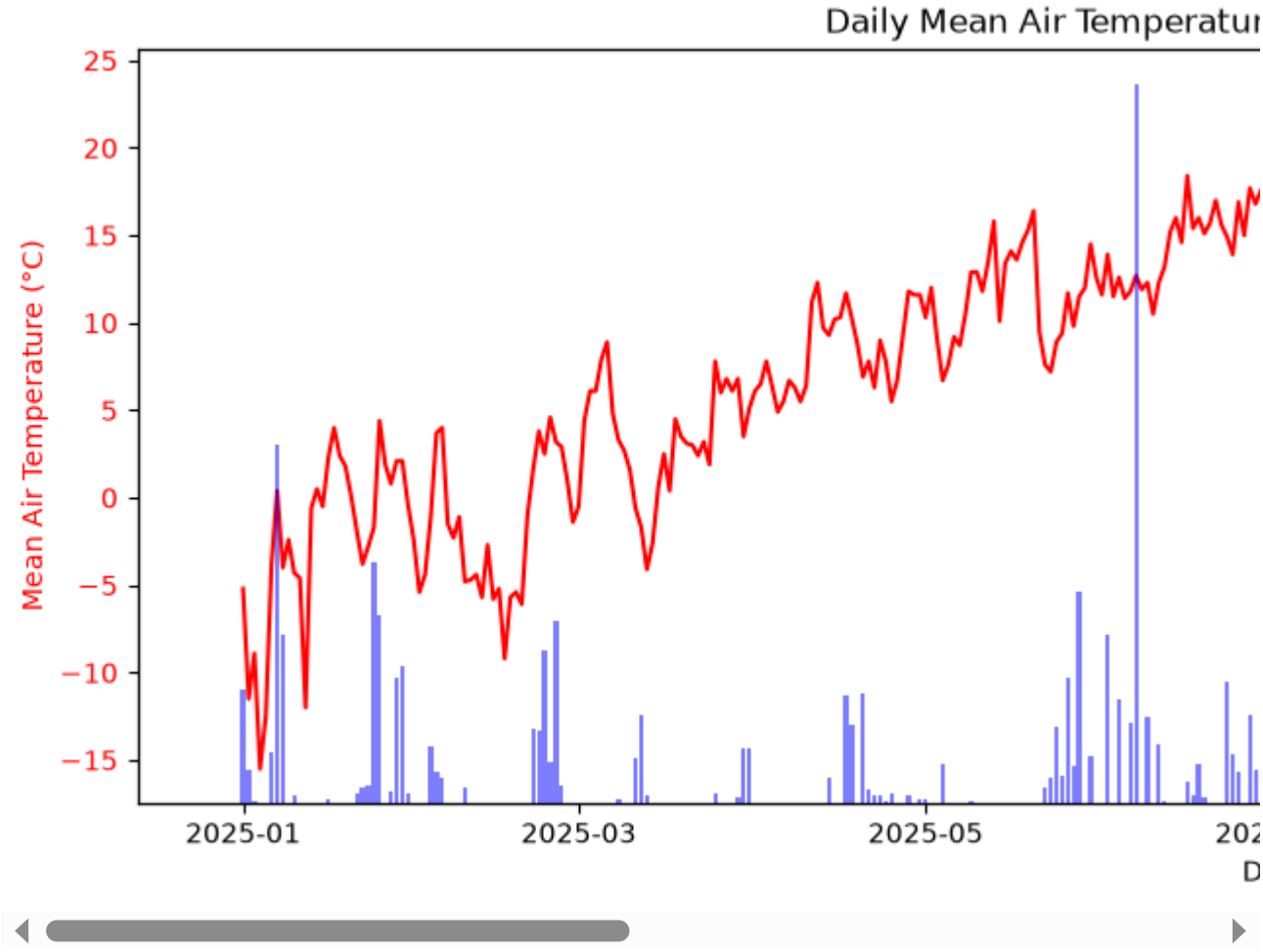
python

```python
fig, ax1 = plt.subplots(figsize=(15, 5))
ax1.set_title('Daily Mean Air Temperature and Precipitation for 2025')
ax1.set_xlabel('Date')

ax1.set_ylabel('Mean Air Temperature (°C)', color='red')
ax1.plot(time_task1, temp_task1, color='red', label='Mean Air Temperat
ax1.tick_params(axis='y', labelcolor='red')

ax2 = ax1.twinx()
ax2.set_ylabel('Precipitation (mm)', color='blue')
ax2.bar(time_task2, precipitation_task2, color='blue', alpha=0.5, labe
ax2.tick_params(axis='y', labelcolor='blue')

plt.show()
```



Task 3

Task 3: This section compares the 2025 temperature series with a 1925 dataset on the same plot and prints summary statistics side by side. The result illustrates how the climate has changed over time, showing the two years in direct comparison and making differences in average level and variability easier to interpret.

python

```python
endpoint = 'https://frost.met.no/observations/v0.jsonld'
parameters = {
'sources': 'SN17850',
'elements': 'mean(air_temperature P1D)',
'referencetime': '1925-01-01/1925-12-31',
}

frost_response = requests.get(endpoint, parameters, auth=(client_id, "
assert frost_response.status_code == 200

frost_data_task3 = frost_response.json()
```

python

```python
temp_task3 = []
time_task3 = []

for entry in frost_data_task3['data']:
    temperature = entry['observations'][0]['value']
    date = pd.to_datetime(entry['referenceTime'])

    temp_task3.append(temperature)
    time_task3.append(date)
```

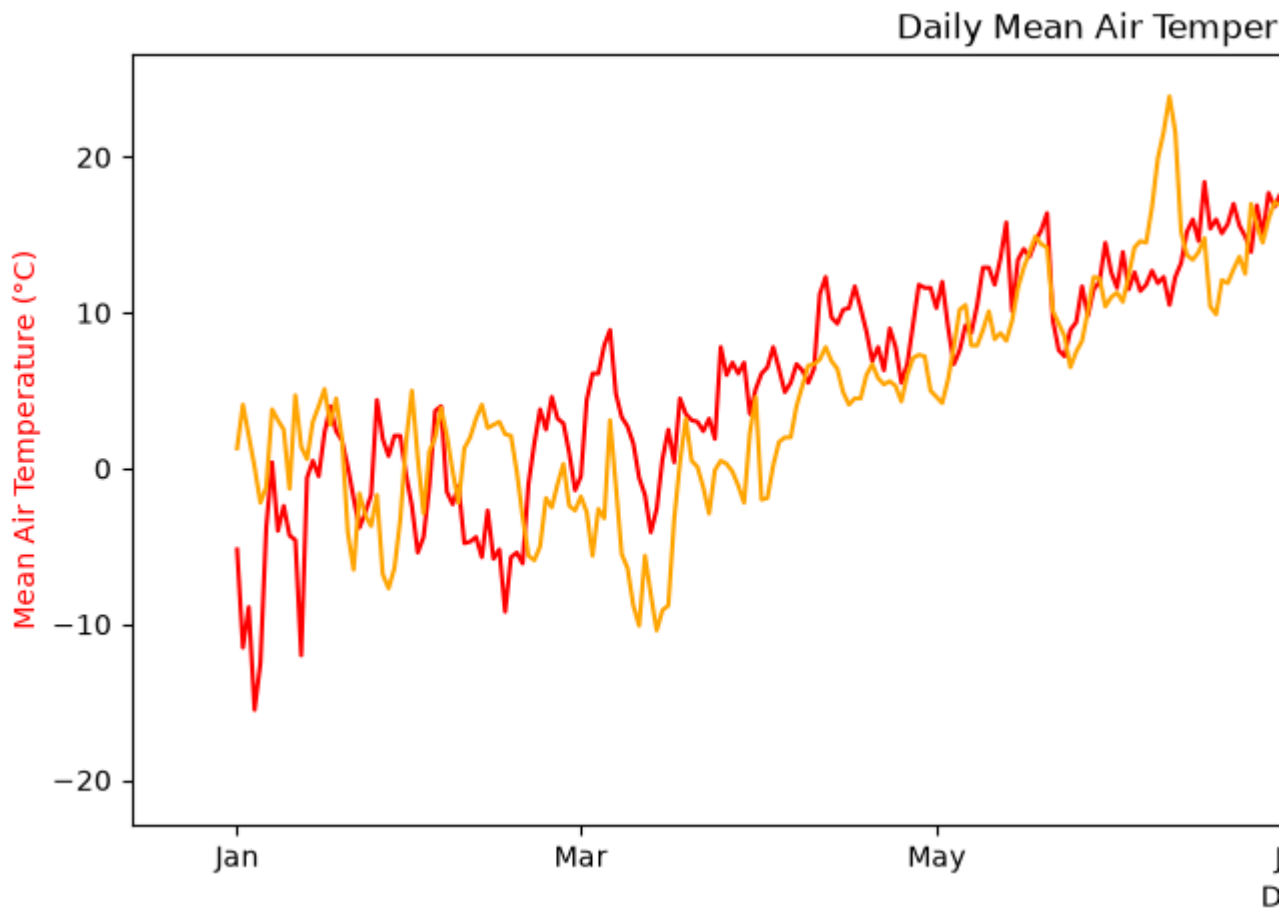
python

```
fig, ax1 = plt.subplots(figsize=(15, 5))
ax1.set_title('Daily Mean Air Temperature for 1925 and 2025')
ax1.set_xlabel('Date')
ax1.set_ylabel('Mean Air Temperature (°C)', color='red')
ax1.plot(time_task1, temp_task1, color='red', label='Mean Air Temperat
ax1.plot(time_task1, temp_task3, color='orange',label='Mean Air Temper

ax1.xaxis.set_major_formatter(mdates.DateFormatter('%b'))

ax1.legend()

plt.show()
```



python

```python
mean_temp_task3 = sum(temp_task3) / len(temp_task3)
max_temp_task3  = max(temp_task3)
min_temp_task3  = min(temp_task3)
median_temp_task3 = st.median(temp_task3)

print("Summary Statistics for Daily Mean Air Temperature in 1925 vs 20
print(f"Mean 1925:   {mean_temp_task3:7.2f} °C  |  Mean 2025:   {mean_
print(f"Max 1925:    {max_temp_task3:7.2f} °C  |  Max 2025:    {max_te
print(f"Min 1925:    {min_temp_task3:7.2f} °C  |  Min 2025:    {min_te
print(f"Median 1925: {median_temp_task3:7.2f} °C  |  Median 2025: {med
```

```
Summary Statistics for Daily Mean Air Temperature in 1925 vs 2025:
Mean 1925:      5.31 °C  |  Mean 2025:      7.90 °C
Max 1925:      24.30 °C  |  Max 2025:      23.70 °C
Min 1925:     -20.70 °C  |  Min 2025:     -15.50 °C
Median 1925:    5.30 °C  |  Median 2025:    8.40 °C
```

Task 4

Task 4: This section defines a heatwave as five or more consecutive days with daily maximum temperature at or above 27 °C, then writes the detected periods to a YAML file. The result is a structured summary of the heatwave dates for the selected station and year, which makes it possible to identify unusually warm periods in a reproducible way.

python

```python
year = 2025
station = 'SN17850'

def find_heatwaves(year, station):
    endpoint = 'https://frost.met.no/observations/v0.jsonld'
    parameters = {
        f'sources': f'{station}',
        'elements': 'max(air_temperature P1D)',
        f'referencetime': f'{year}-01-01/{year}-12-31',
    }

    frost_response = requests.get(endpoint, parameters, auth=(client_i
    assert frost_response.status_code == 200

    frost_data_task4 = frost_response.json()

    temp_task4 = []
    time_task4 = []
    for entry in frost_data_task4['data']:
        temperature = entry['observations'][0]['value']
        date = pd.to_datetime(entry['referenceTime'])

        temp_task4.append(temperature)
        time_task4.append(date)

    counter = 0
    heatwaves = []
    start_date = None
    end_date = None

    for temp, time in zip(temp_task4, time_task4):

        if temp >= 27:
            if counter == 0:
                start_date = time
            counter += 1
            end_date = time
        else:
```

```
            if counter >= 5:
                heatwaves.append({'start_date': start_date, 'end_date'
            counter = 0

    if counter >= 5:
        heatwaves.append({'start_date': start_date, 'end_date': end_da

    return heatwaves


info = (find_heatwaves(year, station))
```

python

```
with open('heatwave_summary.yml', 'w') as file:
    for i in range(len(info)):
        file.write(f"date: \n - {info[i]['start_date']} -> {info[i]['e

    file.write(f"Station ID: {station}\n")
    file.write(f"Station Name: \n")
    file.write(f"Year: {year}\n")
```

Task 5

Task 5: This final section calculates a 7-day moving average from the 2025 temperature data and overlays it on the original daily values. The result smooths short-term fluctuations, making the underlying trend more visible and helping to show periods of sustained warming or cooling that are less obvious in the raw daily series.

python

```python
rolling_average = []
window_size = 7

def calculate_rolling_average(window_size, data):
    for i in range(len(data)):
        if i < window_size - 1:
            rolling_average.append(None)
        else:
            seven_day_window = data[i - window_size + 1:i + 1]
            average = sum(seven_day_window) / window_size
            rolling_average.append(average)

    return rolling_average

calculate_rolling_average(7, temp_task1)
```

```
[None,
 None,
 None,
 None,
 None,
 None,
 -8.142857142857142,
 -7.971428571428571,
 -6.671428571428572,
 -6.014285714285714,
 -4.457142857142856,
 -4.385714285714285,
 -3.928571428571428,
 -3.914285714285714,
 -3.414285714285714,
 -2.742857142857143,
 -1.5571428571428574,
 -0.5571428571428572,
 1.4142857142857141,
 1.5142857142857142,
 1.1714285714285713,
 0.7000000000000001,
 -0.02857142857142845,
 -0.8428571428571427,
 -0.557142857142857,
 -0.5428571428571428,
 -0.4428571428571427,
 0.1285714285714287,
 0.9714285714285715,
 1.3142857142857145,
 1.2142857142857142,
 -0.18571428571428575,
 -1.0857142857142859,
```

-1.3571428571428572,
-1.1285714285714286,
-0.8571428571428573,
-1.0142857142857145,
-1.0,
-0.3857142857142857 3,
-0.4428571428571429,
-0.9571428571428571,
-2.1142857142857143,
-3.5,
-3.6714285714285713,
-4.171428571428572,
-4.757142857142857,
-5.385714285714286,
-5.528571428571429,
-5.671428571428572,
-5.728571428571429,
-5.485714285714286,
-4.428571428571429,
-3.142857142857143,
-1.4714285714285715,
-6.344131569286608e-17,
1.2285714285714284,
2.5142857142857147,
2.8000000000000003,
2.3714285714285714,
1.7571428571428573,
2.0285714285714285,
2.242857142857143,
2.6571428571428575,
3.3714285714285714,
4.5,
5.385714285714286,
5.928571428571429,
5.685714285714285,
5.042857142857143,
4.085714285714286,
2.7142857142857144,
0.8571428571428571,
-0.19999999999999993,
-0.5857142857142856,
-0.6142857142857142,
-0.7857142857142857,
-0.0571428571428571,
0.6857142857142858,
1.7142857142857142,
2.5142857142857147,
2.7714285714285714,
2.8714285714285714,
3.085714285714286,
3.557142857142857,
3.914285714285714,
4.442857142857143,
4.885714285714286,
5.514285714285714,
5.557142857142857,
6.014285714285714,
5.771428571428571,
5.8428571428571425,
5.985714285714286,
6.028571428571429,

5.757142857142857,
6.042857142857143,
6.271428571428571,
6.3,
6.1571428571428575,
5.957142857142857,
6.642857142857143,
7.7,
8.3,
8.671428571428573,
9.228571428571428,
9.914285714285715,
10.671428571428573,
10.542857142857143,
10.042857142857143,
9.642857142857142,
9.428571428571429,
8.871428571428572,
8.685714285714285,
8.128571428571428,
7.442857142857142,
7.142857142857143,
7.485714285714287,
8.057142857142859,
8.814285714285715,
9.185714285714285,
9.542857142857143,
10.471428571428572,
10.814285714285715,
10.442857142857141,
9.842857142857143,
9.5,
9.085714285714285,
9.114285714285714,
9.242857142857144,
9.785714285714286,
10.514285714285714,
11.357142857142858,
12.299999999999999,
12.5,
12.914285714285715,
13.085714285714285,
13.185714285714285,
13.585714285714285,
13.842857142857143,
13.928571428571429,
13.842857142857143,
13.014285714285714,
12.028571428571428,
11.357142857142858,
10.614285714285714,
10.1,
9.157142857142857,
9.442857142857141,
10.071428571428571,
11.114285714285714,
11.642857142857142,
11.957142857142857,
12.27142857142857,
12.514285714285714,
12.671428571428573,

12.585714285714285,
12.200000000000001,
12.214285714285714,
12.257142857142856,
12.028571428571428,
11.885714285714286,
11.842857142857143,
12.1,
12.585714285714285,
13.057142857142859,
13.442857142857141,
14.314285714285715,
15.014285714285714,
15.542857142857143,
15.814285714285715,
15.885714285714286,
16.02857142857143,
16.17142857142857,
15.671428571428573,
15.457142857142857,
15.585714285714285,
15.571428571428571,
15.857142857142858,
15.828571428571427,
16.114285714285714,
16.385714285714286,
16.857142857142858,
16.942857142857143,
16.9,
16.342857142857145,
16.0,
15.842857142857143,
15.528571428571428,
15.514285714285714,
15.385714285714286,
15.871428571428572,
16.771428571428572,
17.942857142857143,
18.571428571428573,
19.242857142857144,
19.82857142857143,
20.428571428571427,
21.071428571428573,
21.457142857142856,
21.47142857142857,
21.87142857142857,
22.357142857142858,
22.599999999999998,
22.685714285714287,
22.7,
22.52857142857143,
22.24285714285714,
21.542857142857144,
20.957142857142856,
19.942857142857143,
19.32857142857143,
18.485714285714288,
18.071428571428573,
17.514285714285716,
17.271428571428572,
16.95714285714286,

16.87142857142857,
16.742857142857144,
16.514285714285712,
16.242857142857144,
16.014285714285712,
15.814285714285713,
15.557142857142855,
15.571428571428571,
15.87142857142857,
16.342857142857145,
16.485714285714288,
16.485714285714288,
16.6,
17.02857142857143,
17.1,
16.257142857142856,
15.442857142857145,
14.571428571428571,
14.185714285714285,
13.714285714285714,
12.942857142857141,
12.571428571428571,
12.799999999999999,
13.057142857142855,
13.37142857142857,
13.857142857142858,
14.285714285714286,
14.985714285714286,
15.385714285714286,
15.700000000000001,
16.014285714285712,
16.242857142857144,
16.2,
15.957142857142857,
15.5,
15.62857142857143,
15.828571428571427,
15.671428571428573,
15.314285714285715,
14.728571428571428,
14.442857142857141,
14.157142857142857,
13.528571428571428,
12.942857142857141,
12.514285714285714,
12.771428571428572,
13.114285714285714,
13.028571428571428,
12.585714285714285,
11.814285714285715,
11.0,
10.342857142857143,
9.414285714285715,
8.685714285714287,
8.442857142857143,
8.871428571428572,
9.185714285714285,
9.214285714285714,
9.442857142857141,
9.728571428571428,
9.6,

9.62857142857143,
9.414285714285715,
9.828571428571427,
10.057142857142859,
9.9,
9.985714285714286,
10.757142857142856,
10.571428571428571,
10.014285714285714,
9.714285714285714,
9.285714285714286,
8.928571428571429,
7.728571428571429,
5.985714285714286,
4.985714285714286,
4.757142857142857,
4.385714285714285,
4.985714285714286,
5.557142857142857,
6.642857142857143,
7.757142857142857,
8.457142857142857,
8.614285714285714,
7.857142857142857,
6.8,
5.985714285714286,
4.671428571428572,
4.314285714285715,
4.342857142857143,
4.557142857142858,
5.9,
7.057142857142857,
8.042857142857143,
9.314285714285715,
9.514285714285714,
9.185714285714285,
8.785714285714286,
8.114285714285716,
7.928571428571429,
7.0,
5.3999999999999995,
4.057142857142857,
3.3,
2.414285714285714,
0.9857142857142855,
-0.942857142857143,
-2.3714285714285714,
-2.942857142857143,
-2.242857142857143,
-1.5142857142857145,
-1.4142857142857144,
-1.1714285714285715,
-0.9142857142857144,
0.6,
2.6142857142857143,
2.728571428571428,
2.314285714285714,
2.914285714285714,
3.9857142857142853,
5.085714285714286,
4.885714285714286,

4.142857142857143,
4.042857142857143,
4.3,
4.557142857142857,
3.814285714285714,
3.6,
3.914285714285714,
3.8857142857142857,
4.057142857142857,
4.6000000000000005,
5.085714285714286,
6.085714285714286,
5.914285714285714,
5.985714285714286,
6.585714285714286,
6.457142857142857,
5.028571428571429,
3.257142857142857,
1.9285714285714286,
0.7142857142857143,
-0.8571428571428571,
-2.3428571428571425,
-2.957142857142857,
-3.242857142857143,
-3.1999999999999997,
-3.4285714285714284]

python

```
fig, ax1 = plt.subplots(figsize=(15, 5))
ax1.set_title('Daily Mean Air Temperature and 7-Day Rolling Average fo
ax1.set_xlabel('Date')
ax1.set_ylabel('Mean Air Temperature (°C)', color='red')
ax1.scatter(time_task1, temp_task1, color='red', label='Mean Air Tempe
ax1.plot(time_task1, rolling_average, color='orange', label='7-Day Rol
ax1.legend()

plt.show()
```

